

Занятие 2. Как редактировать конечную память: decay, delta rule, gates

Главная тема занятия - не ещё один способ сделать attention линейным. Вопрос теперь другой: **как управлять памятью фиксированного размера, если простое бесконечное сложение записей приводит к interference?**

Какой вопрос закрывает занятие

На занятии 1 мы пришли к recurrent representation

$$S_t = S_{t-1} + v_t k_t^\top, \quad y_t = S_t q_t.$$

Здесь $S_t \in \mathbb{R}^{d_v \times d_k}$, поэтому число scalars в persistent state не зависит от длины уже обработанного prefix. Это действительно меняет cost profile autoregressive decode: вместо растущего KV-cache между шагами можно передавать матрицу фиксированного shape.

Но из этого не следует, что проблема длинного контекста решена.

На каждом следующем шаге мы пытаемся поместить новую association в ту же самую матрицу. После десяти tokens и после ста тысяч tokens shape остаётся $d_v \times d_k$. Значит, вопрос хранения постепенно превращается в вопрос **управления ограниченной памятью**.

В этом занятии последовательно появятся три уровня такого управления.

Сначала - **decay**: старые записи постепенно ослабляются.

Затем - **delta rule**: вместо безусловного добавления нового value мы сначала смотрим, что уже хранится по текущему key, и записываем только correction.

Наконец - **gating + delta rule**: одна ручка позволяет глобально ослаблять память, другая - адресно редактировать association.

Получается следующая линия:

additive state $\rightarrow$ decay $\rightarrow$
delta rule $\rightarrow$ gated delta rule.

При этом важно сохранить различие из занятия 1:

- иногда мы меняем сам mathematical operator;
- иногда строим другой algorithm вычисления того же operator;

- иногда только замечаем сходный принцип устройства разных architectures.

Эти три утверждения имеют разную силу. Слово «эквивалентность» к ним без уточнения применять нельзя.

Короткий ресар занятия 1

Дальше нужны две формулы:

$$S_t = S_{t-1} + v_t k_t^\top,$$

$$y_t = S_t q_t.$$

Главный вопрос перед новым материалом: **что произойдёт, если разные записанные keys неортогональны и начинают интерферировать при чтении?**

Chunkwise form и normalized variant здесь заново не выводим: они остаются в занятии 1 как reference. Вместо этого подробнее разобраны геометрия interference и граница между online-learning interpretation внутреннего fast state и внешним обучением model parameters.

Полный notation contract занятия 1

Нужен контракт занятия 1.

Позиции имеют номера $t = 1, \dots, T$.

В коде канонический порядок axes - [B, H, T, D] .

В формулах linear operator q_t и k_t уже находятся после feature map:

$$\phi: \mathbb{R}^{d_h} \rightarrow \mathbb{R}^{d_k}.$$

Размерности:

$$q_t, k_t \in \mathbb{R}^{d_k}, \quad v_t \in \mathbb{R}^{d_v}.$$

Равенство $d_k = d_v$ не предполагается.

Sequence matrices:

$$Q, K \in \mathbb{R}^{T \times d_k}, \quad V, Y \in \mathbb{R}^{T \times d_v}.$$

State orientation фиксирована: $S_t \in \mathbb{R}^{d_v \times d_k}$.

Базовый write и read:

$$S_t = S_{t-1} + v_t k_t^\top, \quad y_t = S_t q_t.$$

C batch и heads state имеет shape $[B, H, d_v, d_k]$.

Causal mask inclusive: $M_{ti} = \mathbf{1}[i \leq t]$.

Поэтому сначала текущая запись входит в S_t , затем вычисляется $y_t = S_t q_t$.

Нужны также три формы operator: $Y = ((QK^\top) \odot M)V$,

$$S_t = S_{t-1} + v_t k_t^\top, \quad y_t = S_t q_t,$$

$$Y_b = Q_b(S_b^{\text{in}})^\top + ((Q_b K_b^\top) \odot M_b)V_b, \quad S_b^{\text{out}} = S_b^{\text{in}} + V_b^\top K_b.$$

Для normalized variant дополнительно используется

$$z_t = z_{t-1} + k_t, \quad y_t = \frac{S_t q_t}{q_t^\top z_t + \varepsilon}.$$

Именно эти orientations и shapes были зафиксированы в контракте занятия 1.

Что должно получаться после чтения

После этого конспекта нужно уметь без дополнительных материалов:

1. Разложить чтение из additive state на полезный сигнал и interference от остальных associations.
2. Объяснить, почему pairwise почти ортогональные keys сами по себе не гарантируют хорошую память при большом числе записей.
3. Сформулировать ограничение capacity через систему $K_m S^\top = V_m$ и объяснить, почему фиксированное число state scalars не позволяет хранить произвольное число независимых associations без потерь.
4. Развернуть recurrence $S_t = \gamma S_{t-1} + v_t k_t^\top$ в weighted causal sum.
5. Прodelать то же для data-dependent γ_t .
6. Записать simplified RetNet-like operator в parallel, recurrent и chunkwise form и проверить, что при $\gamma = 1$ он возвращается к формулам занятия 1.
7. Из quadratic retrieval loss полностью вывести delta update одним шагом gradient descent.
8. Доказать exact correction по направлению текущего key при $\|k_t\|_2 = 1$ и $\beta_t = 1$ и объяснить, почему другие неортогональные directions при этом могут измениться.
9. Объяснить, почему delta rule не является нормализацией linear attention.
10. Проверить shapes в Gated DeltaNet при $d_k \neq d_v$.
11. Отдельно объяснить смысл α_t и β_t и привести ситуацию, где одной из этих ручек недостаточно.
12. Объяснить, какие parallel/recurrent/chunkwise forms существуют у decay и delta operators и почему delta rule parallelization уже нетривиальна.
13. Разместить RetNet, Mamba, Mamba-2/SSD, DeltaNet, Gated DeltaNet, RWKV-7 и Hyena на общей карте, не называя полные architectures эквивалентными.

14. Заменить фразу «fixed state решил проблему длинного контекста» на технически корректное утверждение о cost и capacity.

Словарь занятия

Понятие	English term	Определение
Многозапросное ассоциативное извлечение	multi-query associative recall, MQAR	Задача, где пары ключ-значение разбросаны по последовательности и ключи спрашиваются несколько раз. Нагружает ёмкость state, в отличие от однозапросной версии.
Эффективная длина контекста	effective context length	Длина, до которой заявленное качество ещё воспроизводится. Не совпадает с заявленной длиной окна.
Конечная ассоциативная память	finite associative memory	Memory фиксированного размера, в которой key используется как адрес или направление чтения, а state хранит множество key-value associations в superposition.
Интерференция	interference	Ошибка чтения, возникающая потому, что query взаимодействует не только с нужным key, но и с другими записанными keys.
Коллизия памяти	memory collision	Ситуация, когда две или несколько associations используют одинаковые или сильно перекрывающиеся key directions и поэтому мешают независимому чтению или обновлению.
Ёмкость памяти	memory capacity	Количество и сложность информации, которую state способен сохранять с требуемой точностью. Для фиксированной linear associative memory это не то же самое, что число записанных tokens.
Затухание	decay	Multiplicative ослабление прошлого state перед добавлением новой информации.
Коэффициент сохранения	retention factor	Множитель, определяющий, какая доля старого state переносится на следующий шаг. Обычно обозначается γ или γ_t .
Data-dependent forgetting	data-dependent forgetting	Forgetting, сила которого зависит от текущего input, а не только от расстояния между positions.

Понятие	English term	Определение
Retention	retention	В контексте RetNet - decayed causal operator, допускающий parallel, recurrent и chunkwise recurrent computation.
Delta rule	delta rule	Error-correcting update: новая запись зависит от разницы между желаемым value и тем, что память уже возвращает по текущему key.
Ошибка предсказания	prediction error	Вектор $v_t - \hat{v}_t$, показывающий, насколько текущее содержимое memory отличается от желаемой association.
Targeted correction	targeted correction	Точная коррекция read по направлению текущего key; другие неортогональные directions могут при этом измениться.
Fast weights	fast weights	Параметры-подобное состояние, которое меняется внутри обработки одной sequence. Матрицу S_t удобно интерпретировать как fast weight matrix.
Online update	online update	Update state после каждого нового элемента sequence без отдельного offline fit всей памяти.
Write strength	write strength	Scalar β_t , управляющий силой delta correction.
Forget gate	forget gate	Gate, управляющий сохранением старого state. В Gated DeltaNet эту роль играет α_t .
Gated delta rule	gated delta rule	Update, совмещающий global decay state и delta correction текущей key-value association.
Selective SSM	selective state space model	SSM, параметры state evolution которого зависят от input. Mamba - основной пример следующего занятия.
Generalized delta rule	generalized delta rule	Расширение delta update, где gating, learning rate или erase/write directions могут быть более выразительными, чем один scalar и один key.
Long convolution	long convolution	Sequence operator с convolutional filter, действующим на большой temporal range.
Implicit filter	implicit filter	Filter, коэффициенты которого не хранятся как свободный отдельный parameter на каждый lag, а генерируются компактной parameterization.

1. Почему additive state ломается

Зачем эта тема здесь

В конце занятия 1 мы получили очень привлекательную картину.

Вместо хранения всех прошлых k_i и v_i можно поддерживать

$$S_t = \sum_{i=1}^t v_i k_i^\top.$$

State имеет фиксированный shape. Для autoregressive decode это действительно избавляет от persistent memory, линейно растущей с prefix.

Но update $S_t = S_{t-1} + v_t k_t^\top$ умеет только добавлять.

Он не знает:

- какая старая association больше не нужна;
- записывался ли текущий key раньше;
- надо ли заменить старый value или сложить его с новым;
- насколько state уже переполнен;
- какие старые сведения можно безопасно удалить.

То есть проблема не в том, что state становится больше. Он как раз не становится больше. Проблема в том, что **в одну и ту же конечную матрицу приходит всё больше попыток записи**.

Интуиция

Удобно думать о S как о linear map из key space в value space.

Из-за нашей orientation map читается как $\hat{v} = Sk$.

Если в state записана одна association $S = vk^\top$ и $\|k\|_2 = 1$, то чтение тем же key даёт $Sk = v(k^\top k) = v$.

Пока всё хорошо.

Теперь добавим вторую association: $S = v_1 k_1^\top + v_2 k_2^\top$.

Читаем по k_1 :

$$Sk_1 = v_1(k_1^\top k_1) + v_2(k_2^\top k_1).$$

Если keys нормированы, $Sk_1 = v_1 + (k_1^\top k_2)v_2$.

Второе слагаемое мы не просили. Оно появилось только потому, что k_1 и k_2 не ортогональны.

Это и есть простейшая форма interference.

Понятия, которые вводятся

Finite associative memory - fixed-size state, который связывает keys с values.

Interference - примесь остальных associations при чтении нужной.

Memory collision - особенно сильная interference, когда key directions совпадают или близки.

Capacity - не shape state сам по себе, а способность представлять требуемое множество associations с заданной точностью.

Полезно различать два утверждения.

Первое проверяется буквально: $\#S = d_k d_v$ scalars на head, независимо от T .

Второе уже содержательное: количество независимых associations, которые можно безошибочно восстановить из этих scalars, ограничено.

Формальное изложение

Чтение из additive memory

Перед записью текущего token состояние равно

$$S_{t-1} = \sum_{i=1}^{t-1} v_i k_i^\top.$$

Чтобы не менять orientation занятия 1, будем считать $\hat{v}_t \in \mathbb{R}^{d_v}$ column vector и писать $\hat{v}_t^\top = k_t^\top S_{t-1}^\top$, то есть эквивалентно $\hat{v}_t = S_{t-1} k_t$.

Подставляем additive state:

$$\hat{v}_t^\top = k_t^\top \sum_{i=1}^{t-1} v_i k_i^\top.$$

Матричное умножение линейно по сумме, поэтому

$$\hat{v}_t^\top = \sum_{i=1}^{t-1} k_t^\top k_i v_i^\top.$$

Scalar $k_t^\top k_i$ можно вынести:

$$\hat{v}_t^\top = \sum_{i=1}^{t-1} (k_t^\top k_i) v_i^\top.$$

Теперь допустим, что мы хотим восстановить association с индексом j . Тогда отдельно выделяем её summand:

$$\hat{v}_t^\top = (k_t^\top k_j) v_j^\top + \sum_{\substack{i=1 \\ i \neq j}}^{t-1} (k_t^\top k_i) v_i^\top.$$

Первая часть - desired signal.

Вторая часть - interference:

$$\xi_t^\top = \sum_{\substack{i=1 \\ i \neq j}}^{t-1} (k_t^\top k_i) v_i^\top.$$

Если $k_t = k_j$ и $\|k_j\|_2 = 1$, desired coefficient равен единице: $\hat{v}_t^\top = v_j^\top + \xi_t^\top$.

То есть даже для точного key match правильный value восстанавливается только без ошибки ξ_t , если остальные записанные keys ортогональны текущему key или их contributions каким-то образом взаимно сокращаются.

Когда interference мала

Есть очевидный хороший случай: $k_t^\top k_i = 0$ для всех ненужных i .

Тогда nuisance terms исчезают.

В high-dimensional space случайные нормированные vectors обычно имеют маленький pairwise dot product. Но здесь есть важная ловушка: **маленький каждый summand не означает маленькую их сумму**.

Рассмотрим упрощённую модель. Пусть m unrelated keys являются независимыми isotropic unit vectors, а values имеют unit norm и достаточно независимы, чтобы cross terms в среднем сокращались.

Для случайных unit vectors в $\mathbb{R}^{d_k}$ характерный масштаб

$$\mathbb{E}[(k_t^\top k_i)^2] \approx \frac{1}{d_k}.$$

Если interference содержит примерно m независимых terms, то

$$\mathbb{E}[\|\xi_t\|_2^2] \approx \frac{m}{d_k}.$$

Следовательно, характерный RMS scale interference растёт примерно как

$$\sqrt{\mathbb{E}[\|\xi_t\|_2^2]} \approx \sqrt{\frac{m}{d_k}}.$$

Это не универсальный закон реальной language model - assumptions здесь специально упрощены. Но он показывает важный механизм.

Большой d_k помогает делать отдельные collisions слабее. При этом количество attempts to write растёт вместе с sequence. Если каждая из тысяч маленьких примесей не равна нулю, накопленный noise уже может быть существенным.

Capacity как система linear constraints

Можно сформулировать ограничение ещё жёстче.

Пусть нужно точно сохранить m associations $Sk_i = v_i, \quad i = 1, \dots, m.$

Сложим keys в строки матрицы $K_m \in \mathbb{R}^{m \times d_k}$ и values в $V_m \in \mathbb{R}^{m \times d_v}$.

Тогда все требования одновременно записываются как $K_m S^\top = V_m$.

Если $m \leq d_k$ и rows K_m linearly independent, для произвольного V_m решения могут существовать.

Если же $m > d_k$, то $\text{rank}(K_m) \leq d_k < m$.

Каждый column матрицы $K_m S^\top$ обязан лежать в column space K_m , размерность которого не превосходит d_k .

Но произвольный column V_m является произвольным vector в $\mathbb{R}^m$. При $m > d_k$ он вообще говоря не обязан лежать в этом меньшем subspace.

Значит, **одна linear map S не может точно реализовать произвольное число независимых key-value constraints.**

Это более аккуратная формулировка capacity, чем «после d_k tokens память заканчивается».

Почему последняя фраза слишком сильная? Потому что language model не обязана хранить произвольные независимые pairs. Она может сжимать структуру, распределять информацию между heads и layers, использовать nonlinear computation и не требовать exact lookup для каждого прошлого token.

Но для arbitrary exact associative bindings bottleneck уже виден в одной линейной алгебре.

Что именно фиксировано

На одном head state содержит $d_k d_v$ scalars.

C batch и heads:

S: [B, H, d_v, d_k] .

Количество state scalars не зависит от T .

Количество выполненных writes после T positions равно T на head.

Вот это и есть напряжение:

fixed number of state scalars, growing number of write attempts.

Пример

Возьмём $d_k = 2$, $d_v = 2$.

Пусть

$$k_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

а второй нормированный key

$$k_2 = \begin{bmatrix} 0.8 \\ 0.6 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

После двух additive writes $S_2 = v_1 k_1^\top + v_2 k_2^\top$.

Первый outer product:

$$v_1 k_1^\top = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

Второй:

$$v_2 k_2^\top = \begin{bmatrix} 0 & 0 \\ 0.8 & 0.6 \end{bmatrix}.$$

Итого

$$S_2 = \begin{bmatrix} 1 & 0 \\ 0.8 & 0.6 \end{bmatrix}.$$

Теперь читаем по первому key:

$$S_2 k_1 = \begin{bmatrix} 1 \\ 0.8 \end{bmatrix}.$$

Нужный value был

$$v_1^\top = [1 \quad 0].$$

Ошибка равна

$$[0 \quad 0.8].$$

Откуда она взялась, видно напрямую: $k_1^\top k_2 = 0.8$.

То есть 80% второго value попало в чтение первого.

Additive memory здесь ничего не делает неправильно относительно своей формулы. Именно эта формула и создаёт superposition.

Где здесь легко ошибиться

«Fixed state может хранить бесконечный context, потому что shape не растёт»

Нет.

Он может **принимать input произвольной длины**, не увеличивая shape persistent state. Это не то же самое, что без потерь хранить весь prefix.

Корректная формулировка:

fixed state меняет scaling persistent memory с длиной context, но создаёт compression/capacity bottleneck.

«Если d_k большой, keys почти ортогональны, значит collisions исчезли»

Нет.

Pairwise overlap может быть мал, но число nuisance terms растёт.

Кроме того, learned keys не обязаны быть independent random vectors.

«Capacity равна ровно d_k associations»

Тоже слишком сильно.

d_k возникает как естественная граница для arbitrary independent exact linear key-value constraints. Реальная model может хранить структурированную информацию иначе.

«Interference появляется только после $T > d_k$ »

Нет.

Она появляется уже при двух неортогональных keys.

Условие $m > d_k$ говорит о невозможности generic exact representation произвольных constraints, а не о первом моменте возникновения error.

«Это та же проблема, что KV-cache»

Нет.

KV-cache хранит отдельные historical K/V и растёт по размеру.

Finite state не растёт, поэтому должен сжимать всё больше history в один fixed-size object.

Это разные bottlenecks.

Что теперь понятно

Fixed-size state отвечает на вопрос «как не хранить отдельный K/V для каждого прошлого token».

Он не отвечает на вопрос «что делать, когда state уже содержит много associations».

Additive write не имеет delete, replace или forget operation.

Следующий минимальный шаг - разрешить старому state постепенно исчезать.

2. Чем меряют длинный контекст

Зачем эта тема здесь

Предыдущий раздел показал, что additive state деградирует: чем больше associations, тем сильнее interference. Это утверждение про механизм.

Дальше по курсу будут числа из статей: «поддерживает 1М контекста», «не уступает attention на длинных последовательностях», «лучше на LongBench». Прежде чем их читать, надо понимать, **что**

именно измерено в каждом случае, иначе любая architecture выглядит одинаково хорошо.

Этот раздел нужен по существу: весь остаток курса состоит из сравнений архитектур по опубликованным числам.

Интуиция

Есть разница между тремя утверждениями, и в маркетинговых материалах она обычно стирается:

- модель **принимает** на вход 1M tokens - это свойство заявленного и реализованного окна: такой вход можно подать;
- модель **не ломается** на 1M tokens - выдаёт связный текст, а не мусор;
- модель **использует** информацию из начала этого контекста - извлекает то, что там лежало.

Третье не следует из первых двух. Растущий KV-cache хранит K и V каждой прошлой позиции отдельно и не сжимает prefix в один объект фиксированного размера; использует ли модель эти K и V - отдельный вопрос. В fixed-size state история уже сжата вместе со всем остальным, и вопрос именно в том, насколько сильно она теряется.

Поэтому измерять надо не длину входа, а длину, **на которой качество ещё держится**.

Понятия, которые вводятся

Associative recall - задача «в последовательности были пары ключ-значение, назови значение по ключу». Прямая проверка того, что лежит в памяти.

MQAR, Multi-Query Associative Recall - версия с несколькими запросами в одной последовательности. Ключевое слово - multi.

Needle in a haystack - в длинный нерелевантный текст вставлена одна фраза, по ней задаётся вопрос. Проверка дальнего извлечения одного факта.

Effective context length - длина, до которой заявленное качество ещё воспроизводится, в отличие от заявленной длины окна.

Формальное изложение

Почему одного запроса мало

Возьмём последовательность, в которую разбросаны пары:

Что меряем	Чем	Что устанавливает
нагрузка многими запросами	MQAR, многозапросный recall	как качество падает с числом одновременно нужных пар
устойчивость к длине	перплексия на длинном входе	модель не разваливается на длинном входе
дальнее извлечение одного факта	needle in a haystack	достаёт вставленный факт с данной позиции и расстояния
прикладное качество	наборы задач на длинном контексте	совокупный эффект на конкретном распределении задач

Строки отвечают на разные вопросы, и результат одной не переносится на другую.

Пример

Пусть architecture заявляет окно 1M и показывает 99 % на needle-тесте на всей длине.

Из раздела 1 известно, что state размера $d_v \times d_k$ хранит ограниченное число ассоциаций: при n записях типичная ошибка чтения растёт как $\sqrt{n-1}$. Никакое заявление про окно этого не отменяет.

Значит у такой architecture вполне возможна картина: needle пройден, а многозапросный recall на той же длине с ростом числа пар деградирует. Два числа не противоречат друг другу, они про разное.

Проверяемое следствие: при фиксированном числе state scalars и фиксированной точности нельзя безошибочно хранить сколько угодно независимых пар, поэтому с ростом нагрузки на MQAR у такой architecture надо ожидать деградации. Где именно начнёт падать кривая, зависит не только от размера state, но и от длины и расстояний, словаря, числа запросов и обучения.

Где здесь легко ошибиться

Считать заявленную длину окна измерением. Окно - это то, что модель принимает на вход. Измерение - это то, на какой длине заявленное качество ещё держится.

Сравнивать числа из разных статей напрямую. У MQAR разные реализации расходятся в числе пар, размере словаря и сетке размерностей. Перед тем как называть число с трибуны, надо посмотреть конфигурацию, по которой оно получено.

Переносить вывод с одного теста на другой. «Хорош на needle» не означает «хорош на recall», и наоборот - как показано в таблице выше.

Забывать про подгонку. Открытые наборы задач на длинном контексте попадают в обучающие данные. Для нового benchmark это меньшая проблема, для двухлетнего - большая.

Что теперь понятно

Заявление о длине контекста само по себе ничего не измеряет.

Нагрузку на конечный state многими запросами создаёт многозапросный associative recall, и именно там обычно видна разница между fixed-size state и растущим кэшем.

Needle-тест проверяет дальнейшее извлечение одного факта - слабую форму использования контекста, - и его результат не переносится на многозапросный recall.

Дальше в курсе любое сравнение architectures будет сопровождаться указанием, какой именно вопрос проверялся.

3. Простое забывание

Зачем эта тема здесь

После предыдущего раздела хочется хотя бы не позволять очень старым associations влиять на state вечно.

Самая простая идея - перед новой записью умножать старую память на число меньше единицы:

$$S_t = \gamma S_{t-1} + v_t k_t^\top.$$

Это уже настоящий механизм forgetting.

При этом он очень грубый: одинаковый multiplier применяется ко всему state.

Именно поэтому decay полезен как следующий шаг после additive memory, но ещё не решает задачу targeted editing.

Интуиция

Представим, что $\gamma = 0.9$.

Association, записанная один шаг назад, получает weight 0.9.

Через два шага - 0.9^2 .

Через десять - 0.9^{10} .

Чем старше запись, тем меньше её влияние.

Вместо памяти «всё прошлое с одинаковой долговечностью» получается recency-weighted memory.

Если γ можно выбирать на каждом token, model получает ещё одну возможность: на одном участке сохранить state почти без изменений, а на другом резко его очистить.

Но эта возможность всё равно не адресует конкретную association. Scalar decay не знает, какую часть S породил конкретный key.

Понятия, которые вводятся

Decay - multiplicative ослабление предыдущего state.

Retention factor - multiplier γ .

Data-dependent gating - случай, когда γ_t вычисляется из текущего input.

Effective memory horizon - характерный temporal scale, на котором старые contributions ещё заметны. Это не жёсткое окно, а плавное затухание.

Формальное изложение

Fixed decay

Начинаем с

$$S_t = \gamma S_{t-1} + v_t k_t^\top, \quad 0 \leq \gamma \leq 1.$$

При $S_0 = 0$: $S_1 = v_1 k_1^\top$.

На следующем шаге: $S_2 = \gamma v_1 k_1^\top + v_2 k_2^\top$.

Ещё один шаг:

$$S_3 = \gamma^2 v_1 k_1^\top + \gamma v_2 k_2^\top + v_3 k_3^\top.$$

Pattern теперь виден: запись с position i пережила $t - i$ multiplicative decay steps.

Поэтому

$$S_t = \sum_{i=1}^t \gamma^{t-i} v_i k_i^\top.$$

Проверить формулу можно induction.

Предположим, что для $t - 1$

$$S_{t-1} = \sum_{i=1}^{t-1} \gamma^{t-1-i} v_i k_i^\top.$$

Подставляем в recurrence:

$$S_t = \gamma \sum_{i=1}^{t-1} \gamma^{t-1-i} v_i k_i^\top + v_t k_t^\top.$$

Объединяем степени:

$$S_t = \sum_{i=1}^{t-1} \gamma^{t-i} v_i k_i^\top + v_t k_t^\top.$$

Для текущего $i = t$ коэффициент равен $\gamma^{t-t} = 1$.

Значит,

$$S_t = \sum_{i=1}^t \gamma^{t-i} v_i k_i^\top.$$

Теперь читаем текущим query после update: $y_t = S_t q_t$.

Подставляем state:

$$y_t = \sum_{i=1}^t \gamma^{t-i} (q_t^\top k_i) v_i.$$

Получился causal pairwise operator, в котором similarity $q_t^\top k_i$ дополнительно умножается на temporal factor γ^{t-i} .

Half-life

Если хочется интерпретировать γ не как абстрактный parameter, а как temporal scale, удобно найти расстояние h , после которого contribution уменьшается вдвое:

$$\gamma^h = \frac{1}{2}.$$

Берём logarithm:

$$h = \frac{\log(1/2)}{\log \gamma}.$$

Например, при $\gamma = 0.99$ $h \approx 69$.

При $\gamma = 0.999$ $h \approx 693$.

Это не означает, что память обрывается на 69 или 693 tokens. Вес просто уменьшается экспоненциально.

Почему state magnitude может стабилизироваться

Допустим, для каждого write $\|v_i k_i^\top\|_F \leq C$.

Тогда по triangle inequality

$$\|S_t\|_F \leq \sum_{i=1}^t \gamma^{t-i} \|v_i k_i^\top\|_F.$$

Следовательно,

$$\|S_t\|_F \leq C \sum_{r=0}^{t-1} \gamma^r.$$

При $0 \leq \gamma < 1$ geometric series ограничена:

$$\sum_{r=0}^{t-1} \gamma^r \leq \frac{1}{1-\gamma}.$$

Поэтому

$$\|S_t\|_F \leq \frac{C}{1-\gamma}.$$

Это полезное свойство decay: старые terms не обязаны бесконечно наращивать norm state.

Но отсюда **не следует отсутствие interference**. Ограниченная norm и корректная associative memory - разные свойства.

Data-dependent decay

Теперь разрешим retention factor зависеть от position: $S_t = \gamma_t S_{t-1} + v_t k_t^\top$.

Разворачиваем первые шаги: $S_1 = v_1 k_1^\top$, $S_2 = \gamma_2 v_1 k_1^\top + v_2 k_2^\top$,

$$S_3 = \gamma_3 \gamma_2 v_1 k_1^\top + \gamma_3 v_2 k_2^\top + v_3 k_3^\top.$$

Contribution записи i умножается на все gates, встретившиеся после неё.

Поэтому

$$S_t = \sum_{i=1}^t \left(\prod_{j=i+1}^t \gamma_j \right) v_i k_i^\top.$$

Для $i = t$ product пустой, по соглашению он равен единице.

Read:

$$y_t = \sum_{i=1}^t \left(\prod_{j=i+1}^t \gamma_j \right) (q_t^\top k_i) v_i.$$

В отличие от fixed γ , temporal weight теперь зависит не только от distance $t - i$, но и от tokens между i и t .

Если на некотором шаге γ_j близко к нулю, почти вся информация из более далёкого prefix резко подавляется.

Пример

Пусть $d_k = d_v = 1$, все keys и queries равны единице: $k_t = q_t = 1$.

Values: $v_1 = 2$, $v_2 = 4$, $v_3 = 8$.

Возьмём $\gamma = 0.5$.

Первый шаг: $S_1 = 2$.

Второй: $S_2 = 0.5 \cdot 2 + 4 = 5$.

Третий: $S_3 = 0.5 \cdot 5 + 8 = 10.5$.

Через unrolled form получаем то же: $S_3 = 0.5^2 \cdot 2 + 0.5 \cdot 4 + 8 = 0.5 + 2 + 8 = 10.5$.

Старый v_1 всё ещё присутствует, но с weight **0.25**.

Теперь предположим, что второй gate data-dependent и равен **0.01**. Тогда почти всё, что было до второго token, резко подавится.

Такой gate можно интерпретировать как learned reset-like event.

Где здесь легко ошибиться

«Decay удаляет collision»

Нет.

Если два values записаны по одному и тому же key, старый просто получает меньший temporal weight. Он не удаляется именно потому, что новый key совпал со старым.

Decay различает **возраст**, но не identity association.

«Если γ_t зависит от input, мы получили Mamba»

Нет.

Data-dependent scalar retention - один design principle.

Mamba 2023 является selective SSM: input управляет параметрами state-space dynamics, а сама architecture включает конкретную SSM parameterization и hardware-aware computation. Авторы формулируют selectivity именно как input-dependent SSM parameters.

Поэтому $S_t = \gamma_t S_{t-1} + v_t k_t^\top$ может быть полезной моделью идеи selective forgetting, но одной этой строкой Mamba не определяется.

Mamba-2/SSD дальше формализует связи между определёнными SSM и attention-like structured operators, но это отдельный результат следующего занятия.

«Decay решил capacity»

Тоже нет.

Он создаёт turnover: старые contributions постепенно освобождают влияние на state.

Но если одновременно нужны очень многие long-lived associations, decay заставит выбирать между двумя плохими режимами:

- γ близко к единице - long-term information живёт дольше, но interference накапливается;
- меньший γ - interference старых записей меньше, но полезная long-term information тоже исчезает.

«Ограниченная norm state означает хорошую память»

Нет.

Можно иметь state с маленькой norm и очень плохим retrieval.

Что теперь понятно

Decay добавляет memory lifetime.

Fixed γ задаёт примерно exponential recency bias.

Data-dependent γ_t позволяет менять скорость forgetting по содержимому sequence.

Но это всё ещё **global operation** над прошлым state.

Чтобы сказать «по этому конкретному key раньше лежало одно значение, теперь хочу другое», нужен update, который сначала посмотрит на текущее содержимое memory.

4. RetNet как чистый пример decay

Зачем эта тема здесь

RetNet нужен в этом курсе не потому, что нужно подробно изучить ещё одну architecture 2023 года.

Он удобен по другой причине: retention mechanism очень явно показывает тот же принцип, который был центральным на занятии 1.

Один decayed operator можно вычислять разными algorithms.

Авторы RetNet прямо выделяют parallel, recurrent и chunkwise recurrent computation paradigms.

Поэтому RetNet хорошо связывает две недели:

- на первой мы увидели три формы additive linear operator;
- сейчас просто добавим temporal decay и посмотрим, как изменятся все три.

Полную RetNet architecture здесь восстанавливать не нужно. Нас интересует simplified real-valued single-head core.

Интуиция

В additive operator pairwise weight равен $q_t^\top k_i$.

В decayed operator он становится $\gamma^{t-i} q_t^\top k_i$.

То есть каждая пара positions получает два множителя:

- content similarity;

- temporal retention.

В parallel form эти weights можно сразу записать в matrix.

В recurrent form exponential factor возникает автоматически: каждый шаг умножает весь предыдущий state на γ .

В chunkwise form часть decay происходит между chunks, а часть - внутри chunk.

Это всё один operator, если formulas совпадают точно.

Понятия, которые вводятся

Retention - causal pairwise interaction с temporal decay.

Parallel retention - вычисление pairwise retention weights на sequence целиком.

Recurrent retention - вычисление того же simplified operator через state recurrence.

Chunkwise recurrent retention - обработка текущего chunk matrix operations с recurrent state между chunks.

Формальное изложение

Decayed causal mask

Для одного head с fixed γ определим matrix

$$R_{ti}^{(\gamma)} = \mathbf{1}[i \leq t] \gamma^{t-i}.$$

При $\gamma = 1$ $R_{ti}^{(1)} = M_{ti}$.

То есть обычный inclusive causal mask занятия 1 является частным случаем retention mask.

Parallel form

Pairwise operator:

$$y_t = \sum_{i=1}^t \gamma^{t-i} (q_t^\top k_i) v_i.$$

Матрица similarities: $QK^\top \in \mathbb{R}^{T \times T}$.

Elementwise умножаем её на temporal retention matrix: $(QK^\top) \odot R^{(\gamma)}$.

И агрегируем values: $Y = ((QK^\top) \odot R^{(\gamma)})V$.

Shapes остаются: $Y \in \mathbb{R}^{T \times d_v}$.

Это literal parallel form.

Он точный, но по-прежнему содержит dense object $T \times T$. Поэтому факт существования parallel representation сам по себе не означает linear memory/work в T .

Recurrent form

Берём $S_t = \gamma S_{t-1} + v_t k_t^\top$.

При $S_0 = 0$ мы уже получили

$$S_t = \sum_{i=1}^t \gamma^{t-i} v_i k_i^\top.$$

Read: $y_t = S_t q_t$.

Подстановка даёт

$$y_t = \sum_{i=1}^t \gamma^{t-i} (q_t^\top k_i) v_i.$$

Это ровно тот же operator, который был определён parallel form.

Здесь «эквивалентность» означает **равенство outputs при одинаковых Q, K, V и γ** , а не просто сходство идеи.

Chunkwise form

Пусть текущий chunk b имеет длину c_b .

Incoming state находится до первой position chunk:

$$S_b^{\text{in}} \in \mathbb{R}^{d_v \times d_k}.$$

Для local positions $r, s = 1, \dots, c_b$ определим intra-chunk retention matrix

$$R_{b,rs}^{(\gamma)} = \mathbf{1}[s \leq r] \gamma^{r-s}.$$

Contribution incoming state к local position r переживает r decay steps.

Поэтому введём diagonal matrix

$$\Lambda_b = \text{diag}(\gamma, \gamma^2, \dots, \gamma^{c_b}).$$

Тогда contribution history до chunk ко всем outputs равен $\Lambda_b Q_b (S_b^{\text{in}})^\top$.

Внутри chunk используется обычная pairwise similarity, но causal mask заменяется decayed mask: $((Q_b K_b^\top) \odot R_b^{(\gamma)}) V_b$.

Получаем exact chunkwise output:

$$Y_b = \Lambda_b Q_b (S_b^{\text{in}})^\top + ((Q_b K_b^\top) \odot R_b^{(\gamma)}) V_b.$$

Теперь нужен state после всего chunk.

Incoming state переживает c_b decay steps: $\gamma^{c_b} S_b^{\text{in}}$.

Первая запись внутри chunk переживает ещё $c_b - 1$ steps, вторая - $c_b - 2$, последняя - ноль.

Определим

$$W_b = \text{diag}(\gamma^{c_b-1}, \gamma^{c_b-2}, \dots, 1).$$

Тогда

$$S_b^{\text{out}} = \gamma^{c_b} S_b^{\text{in}} + V_b^\top W_b K_b.$$

Проверка граничного случая

Положим $\gamma = 1$.

Тогда

$$\Lambda_b = I, \quad W_b = I, \quad R_b^{(1)} = M_b.$$

Формулы превращаются в контракт занятия 1:

$$Y_b = Q_b (S_b^{\text{in}})^\top + ((Q_b K_b^\top) \odot M_b) V_b,$$

$$S_b^{\text{out}} = S_b^{\text{in}} + V_b^\top K_b.$$

Это полезный structural test derivation.

Что меняется при γ_t

Для data-dependent scalar decay pairwise retention между positions i и t равен

$$\prod_{j=i+1}^t \gamma_j.$$

Поэтому formal parallel weights существуют:

$$R_{ti} = \mathbf{1}[i \leq t] \prod_{j=i+1}^t \gamma_j.$$

Recurrent form тоже остаётся простой: $S_t = \gamma_t S_{t-1} + v_t k_t^\top$.

Chunkwise form остаётся exact, но вместо степеней γ^r нужны prefix и suffix products gates внутри chunk.

То есть data dependence ещё не уничтожает три формы. Но implementation становится менее симметричным.

Пример

Снова возьмём scalar case:

$$q_t = k_t = 1, \quad v_1 = 2, \quad v_2 = 4, \quad v_3 = 8, \quad \gamma = 0.5.$$

В recurrent form: $S_1 = 2, S_2 = 0.5 \cdot 2 + 4 = 5, S_3 = 0.5 \cdot 5 + 8 = 10.5$.

Теперь третий row parallel retention weights:

$$[\gamma^2 \quad \gamma \quad 1] = [0.25 \quad 0.5 \quad 1].$$

Умножаем на values: $0.25 \cdot 2 + 0.5 \cdot 4 + 1 \cdot 8 = 10.5$.

Теперь chunkwise computation с первым chunk из двух positions.

После первого chunk: $S_2 = 5$.

Во втором chunk только один token. Incoming state получает factor 0.5: $S_3 = 0.5 \cdot 5 + 8 = 10.5$.

Все три computations дают один результат не потому, что они похожи, а потому, что являются algebraic rearrangements одного simplified decayed operator.

Где здесь легко ошибиться

«RetNet - это просто формула $S_t = \gamma S_{t-1} + v_t k_t^\top$ »

Нет.

Мы используем эту формулу как simplified real-valued core для понимания retention.

Полная RetNet architecture содержит дополнительные positional и multi-scale design choices. Статья важна здесь прежде всего тем, что явно строит parallel, recurrent и chunkwise recurrent representations

retention mechanism.

«Parallel form означает дешёвую по T training»

Нет.

Literal matrix $R(\gamma) \in \mathbb{R}^{T \times T}$ остаётся quadratic-size object.

Parallelism и asymptotic work - разные coordinates cost.

«Если operator имеет recurrent form, его всегда легко эффективно обучать»

Нет.

Recurrent decode и hardware-efficient full-sequence training предъявляют разные требования.

Эта проблема станет особенно заметна у delta rule.

«RetNet и Mamba эквивалентны, потому что оба умеют decay»

Нет.

Общее наличие state decay - сходство design principle.

Для Mamba используется другая SSM parameterization и другой набор architectural components.

Что теперь понятно

Decay не мешает существованию parallel, recurrent и chunkwise forms.

Для scalar exponential decay связь особенно прозрачна: temporal mask γ^{t-i} и repeated multiplication state на γ являются двумя представлениями одного operator.

Следующий update будет сложнее. Вместо scalar decay state начнёт изменяться matrix transformation, зависящей от текущего key.

5. Delta rule: записываем ошибку, а не новый value целиком

Зачем эта тема здесь

Декау отвечает на вопрос:

насколько старой памяти разрешить пережить текущий шаг?

Но он не отвечает на другой вопрос:

что делать, если текущий key уже присутствует в memory и новое значение должно заменить старое?

Представим:

Alice lives in Rome.

Позже в sequence появляется:

Alice moved to Paris.

Если обе записи используют похожий key direction, additive memory склонна получить superposition Rome + Paris.

Декау может сделать Rome слабее только потому, что запись старше.

Delta rule использует более точную информацию: перед записью Paris она спрашивает у самой memory, что уже возвращается для current key, и записывает correction.

Именно поэтому delta rule - центральный механизм этого занятия.

Интуиция

Пусть state сейчас считает, что по key k_t хранится value $\hat{v}_t = S_{t-1}k_t$.

А мы хотим получить v_t .

Ошибка: $e_t = v_t - \hat{v}_t$.

Additive rule пишет весь v_t : $S_t = S_{t-1} + v_t k_t^\top$.

Delta rule пишет только correction: $S_t = S_{t-1} + \beta_t e_t k_t^\top$.

Если memory уже возвращает правильный value, то $e_t = 0$ и ничего добавлять не требуется.

Если memory возвращает неправильное значение, update двигает её именно в сторону текущей retrieval error.

Это очень похоже на online learning. На самом деле это не только аналогия: update буквально получается как один gradient descent step по quadratic retrieval loss.

Понятия, которые вводятся

Delta rule - error-correcting update associative memory.

Prediction error - $v_t - \hat{v}_t$.

Write strength β_t - величина correction, обычно между нулём и единицей.

Targeted correction - свойство, при котором read по конкретному нормированному key можно точно скорректировать до нового target; другие неортогональные directions могут измениться.

Fast weight matrix - S_t , рассматриваемая как linear model, параметры которой обновляются внутри forward pass.

Online learning interpretation - взгляд, при котором каждый token создаёт маленькую regression problem для current state.

Delta rule для fast-weight linear Transformers появился до modern DeltaNet scaling work: Schlag, Irie и Schmidhuber в 2021 году сформулировали finite-memory/fast-weight interpretation и delta-like correction rule. Современная работа Yang et al. 2024 показала hardware-efficient sequence parallelization DeltaNet через memory-efficient representation products of generalized Householder transformations.

Формальное изложение

Порядок операций

Здесь появляется дополнительный internal read.

До update имеется S_{t-1} .

Сначала текущим key читаем **старую** association: $\hat{v}_t = S_{t-1}k_t$.

Затем вычисляем error: $e_t = v_t - \hat{v}_t$.

После этого обновляем state: $S_t = S_{t-1} + \beta_t e_t k_t^\top$.

И только после update выполняется обычный output read текущей position: $y_t = S_t q_t$.

Это совместимо с inclusive semantics занятия 1: current write уже входит в S_t , из которого читается y_t .

Delta update

Подставляем error: $S_t = S_{t-1} + \beta_t(v_t - \hat{v}_t)k_t^\top$.

Поскольку $\hat{v}_t = S_{t-1}k_t$, получаем

$$S_t = S_{t-1} + \beta_t(v_t - S_{t-1}k_t)k_t^\top.$$

Транспонируем выражение внутри parentheses:

$$(v_t - S_{t-1}k_t)^\top = v_t^\top - k_t^\top S_{t-1}^\top.$$

Поэтому

$$S_t = S_{t-1} + \beta_t v_t k_t^\top - \beta_t S_{t-1} k_t k_t^\top.$$

Собираем terms с S_{t-1} :

$$S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Здесь $I \in \mathbb{R}^{d_k \times d_k}$.

Проверим shapes.

Outer product: $k_t k_t^\top \in \mathbb{R}^{d_k \times d_k}$.

Следовательно,

$$S_{t-1}(I - \beta_t k_t k_t^\top) \in \mathbb{R}^{d_v \times d_k}.$$

Новая запись: $v_t k_t^\top \in \mathbb{R}^{d_v \times d_k}$.

Оба summands имеют тот же shape, что state.

Равенство $d_k = d_v$ нигде не понадобилось.

Вывод как gradient descent

Теперь выведем ту же формулу не из интуиции, а из optimization problem.

Рассматриваем state S как parameters linear predictor $f_S(k) = Sk$.

На текущем pair (k_t, v_t) хотим уменьшить squared retrieval error:

$$\ell_t(S) = \frac{1}{2} \|Sk_t - v_t\|_2^2.$$

Обозначим $e = Sk_t - v_t \in \mathbb{R}^{d_v}$.

Тогда

$$\ell_t(S) = \frac{1}{2} \sum_{b=1}^{d_v} e_b^2.$$

Каждая component error равна

$$e_b = \sum_{a=1}^{d_k} S_{ba}(k_t)_a - (v_t)_b.$$

Первый индекс у S - координата value, второй - координата key. Найдём derivative по одному element S_{ba} .

Сначала

$$\frac{\partial \ell_t}{\partial S_{ba}} = e_b \frac{\partial e_b}{\partial S_{ba}}.$$

Из определения e_b :

$$\frac{\partial e_b}{\partial S_{ba}} = (k_t)_a.$$

Поэтому

$$\frac{\partial \ell_t}{\partial S_{ba}} = e_b (k_t)_a.$$

Matrix с elements $e_b (k_t)_a$ является outer product: $\nabla_S \ell_t = e k_t^\top$, shape $d_v \times d_k$, как у S .

Возвращаем определение error: $\nabla_S \ell_t = (S k_t - v_t) k_t^\top$.

Теперь делаем один gradient descent step из старого state: $S_t = S_{t-1} - \beta_t \nabla_S \ell_t(S_{t-1})$.

Подставляем gradient:

$$S_t = S_{t-1} - \beta_t (S_{t-1} k_t - v_t) k_t^\top.$$

Меняем знак внутри error:

$$S_t = S_{t-1} + \beta_t (v_t - S_{t-1} k_t) k_t^\top.$$

Получили в точности delta rule.

То есть β_t можно интерпретировать как learning rate **для fast state внутри sequence**.

Это не learning rate внешнего AdamW, которым обучаются обычные model parameters. Здесь gradient step является частью forward computation.

Exact overwrite

Теперь докажем ключевое свойство.

До update current key возвращает $\hat{v}_t = S_{t-1}k_t$.

После update: $S_t = S_{t-1} + \beta_t(v_t - \hat{v}_t)k_t^\top$.

Читаем новым state по тому же key:

$$S_t k_t = S_{t-1} k_t + \beta_t (v_t - \hat{v}_t) k_t^\top k_t.$$

Первый term - $\hat{v}_t$:

$$S_t k_t = \hat{v}_t + \beta_t \|k_t\|_2^2 (v_t - \hat{v}_t).$$

Если key нормирован, $\|k_t\|_2 = 1$, то $S_t k_t = (1 - \beta_t)\hat{v}_t + \beta_t v_t$.

Это очень полезная форма.

При $\beta_t = 0$: $S_t k_t = \hat{v}_t$.

Association не меняется.

При $0 < \beta_t < 1$ получаем interpolation между старым и новым value.

При $\beta_t = 1$: $S_t k_t = v_t$.

Это exact correction по направлению текущего key.

После одного update память возвращает ровно новый target по текущему нормированному key.

Зачем нормировать key

Если $\|k_t\|_2^2 \neq 1$, то фактическая correction strength становится $\beta_t \|k_t\|_2^2$.

Например, при $\beta_t = 1$ и norm squared 4 мы не получим interpolation coefficient 1, а сделаем слишком большой step.

Теоретически exact one-step correction можно получить, выбрав

$$\beta_t = \frac{1}{\|k_t\|_2^2},$$

если $k_t \neq 0$.

В этом алгебраическом утверждении β_t рассматривается как произвольный положительный step.

Если implementation дополнительно ограничивает $\beta_t \in [0, 1]$, одношаговая exact correction для ненормированного key не гарантируется: значение $1/\|k_t\|_2^2$ может оказаться больше 1. При $\|k_t\|_2 = 1$ достаточно $\beta_t = 1$.

Но L2-normalized keys дают более стабильную и понятную interpretation β_t как write strength.

Насколько update действительно targeted

Фраза targeted correction иногда создаёт впечатление, что все остальные associations вообще не меняются.

Это верно только для directions, ортогональных текущему key.

Возьмём другой query direction u . После delta update

$$S_t u = S_{t-1} u + \beta_t (v_t - \hat{v}_t) (k_t^\top u).$$

Если $u^\top k_t = 0$, то его read не изменится.

Если u пересекается с k_t , update затронет и его.

Поэтому delta rule адреснее scalar decay, но finite-dimensional key geometry никуда не исчезает. Collisions между похожими keys всё ещё важны.

Delta rule как erase + write

Matrix form

$$S_t = S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

удобно читать как две операции.

Первая: $S_{t-1} (I - \beta_t k_t k_t^\top)$.

Она уменьшает component старого state вдоль k_t .

Вторая: $\beta_t v_t k_t^\top$.

Она записывает новый value в том же key direction.

При unit key matrix $I - \beta_t k_t k_t^\top$ имеет eigenvalue $1 - \beta_t$ вдоль k_t и eigenvalue 1 во всех ортогональных directions.

То есть старый state подавляется адресно вдоль текущего key.

Пример 1: повторная запись одного key

Самый чистый пример - scalar key space: $d_k = d_v = 1$, $k_1 = k_2 = 1$.

Пусть сначала записано $v_1 = 1$.

Additive state после первого token: $S_1 = 1$.

Теперь приходит новая association по тому же key: $v_2 = 3$.

Additive rule

$$S_2 = S_1 + v_2 = 4.$$

Read по key **1** даёт **4**.

Это не старое значение **1** и не новое **3**, а сумма.

Delta rule

Сначала читаем старое: $\hat{v}_2 = 1$.

Error: $v_2 - \hat{v}_2 = 2$.

При $\beta_2 = 1$: $S_2 = 1 + 1 \cdot 2 = 3$.

Read: $S_2 k_2 = 3$.

Read по направлению текущего key точно скорректирован до нового target; другие неортогональные directions могли измениться.

Пример 2: чем delta rule отличается от нормализации

Возьмём тот же пример: $k_1 = k_2 = q = 1$, $v_1 = 1$, $v_2 = 3$.

Для normalized additive linear attention после двух writes: $S_2 = 1 + 3 = 4$.

Normalizer state: $z_2 = 1 + 1 = 2$.

Ignoring ϵ для этого численного примера:

$$y = \frac{4}{2} = 2.$$

Нормализатор превратил сумму в average-like quantity.

Но новый value был **3**.

Delta rule при $\beta = 1$ дал $S_2 = 3$.

То есть механизмы принципиально разные.

Normalization отвечает примерно на вопрос:

на какой total kernel mass поделить агрегированный numerator?

Delta rule отвечает на вопрос:

что memory уже возвращает по current key и какое correction нужно записать, чтобы вернуть target?

Normalization не вычисляет prediction error $v - \hat{v}$ и не удаляет старую association из S .

Пример 3: collision всё ещё имеет значение

Вернёмся к двумерному state из первого раздела:

$$S = \begin{bmatrix} 1 & 0 \\ 0.8 & 0.6 \end{bmatrix}.$$

По key

$$k = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

memory возвращает

$$\hat{v} = \begin{bmatrix} 1 \\ 0.8 \end{bmatrix}.$$

Пусть теперь хотим записать

$$v = \begin{bmatrix} 2 \\ 0 \end{bmatrix}.$$

Error:

$$v - \hat{v} = \begin{bmatrix} 1 \\ -0.8 \end{bmatrix}.$$

При $\beta = 1$ update добавит

$$(v - \hat{v})k^{\top} = \begin{bmatrix} 1 & 0 \\ -0.8 & 0 \end{bmatrix}.$$

Новый state:

$$S' = \begin{bmatrix} 2 & 0 \\ 0 & 0.6 \end{bmatrix}.$$

По текущему key:

$$S'k = \begin{bmatrix} 2 \\ 0 \end{bmatrix}.$$

Exact overwrite сработал.

Но мы одновременно удалили ту часть второй association, которая раньше leaking into direction k .

Это ожидаемо: один delta-шаг правит mapping только по текущему direction и задевает все неортогональные. При этом сами две associations здесь совместимы: keys линейно независимы, и точное состояние существует, например $S_* = \begin{bmatrix} 2 & -8/3 \\ 0 & 5/3 \end{bmatrix}$. К нему сходятся delta-правки по очереди по k и по второму key - это метод Качмажа для линейных систем. Проблема в локальности одного update, а не в неразрешимости constraints.

Где здесь легко ошибиться

«Delta rule - это normalized linear attention»

Нет.

Normalizer вводит дополнительный state z_t и делит read на scalar.

Delta rule меняет **write dynamics самого S_t** в зависимости от текущего retrieval error.

«Delta rule гарантирует, что другие keys не изменятся»

Только ортогональные текущему key directions не меняются.

Для другого direction u изменение пропорционально $u^\top k_t$.

«Delta rule полностью решил finite capacity»

Нет.

Он существенно улучшает memory editing при key reuse, но S_t всё ещё finite-dimensional.

«Gradient descent здесь означает, что на inference запускается обычный optimizer по network weights»

Нет.

Gradient step - математическая форма state update.

Slow parameters, которые производят k_t, v_t, β_t и другие quantities, по-прежнему обучаются обычным outer training process.

«При $\beta_t = 1$ exact correction работает для любого key»

Только с соответствующей normalization или корректно выбранным step size.

В нашей основной формулировке exact correction по направлению текущего key утверждается при $\|k_t\|_2 = 1$.

«Delta update и additive update отличаются только coefficient перед write»

Нет.

Ключевой term - $-\beta_t S_{t-1} k_t k_t^\top$.

Он зависит от старого state и выполняет erase/correction.

Есть ли у DeltaNet три формы

Recurrent form очевидна:

$$S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Обозначим transition $A_t = I - \beta_t k_t k_t^\top$.

Тогда $S_t = S_{t-1} A_t + \beta_t v_t k_t^\top$.

Unroll даёт

$$S_t = S_0 A_1 A_2 \cdots A_t + \sum_{i=1}^t \beta_i v_i k_i^\top A_{i+1} A_{i+2} \cdots A_t.$$

При $S_0 = 0$ остаётся только сумма.

Следовательно, contribution старого value v_i к output y_t имеет coefficient $\beta_i k_i^\top A_{i+1} \cdots A_t q_t$.

Формальная pairwise representation поэтому существует.

Но здесь есть качественное отличие от fixed decay.

Для decay transition - scalar γI , поэтому product сразу превращается в простое число γ^{t-i} .

Для delta rule каждый A_j зависит от k_j и является matrix.

Эти matrices вообще говоря не commute, и pairwise coefficient уже содержит ordered product state transitions.

Поэтому простой cheap expression вида $((QK^T) \odot M)V$ больше не получается напрямую из raw Q, K, V .

Yang et al. 2024 показывают, как использовать структуру rank-one transitions и WY representation для hardware-efficient chunkwise DeltaNet. В их formulation после построения специальных pseudo-values дальнейшая часть снова напоминает linear attention, но вычислить эти pseudo-values эффективно - как раз нетривиальная часть algorithm.

Итого:

- recurrent form есть;
- exact chunkwise form есть;
- formal sequence-level pairwise/parallel representation есть;
- **наивная fully parallel computation уже не является дешёвой автоматически.**

Это первый явный мост к занятию 3: наличие красивой recurrence ещё ничего не говорит о hardware-efficient training algorithm.

Что теперь понятно

Delta rule превращает finite state из write-only accumulator в editable associative memory.

Центральная формула

$$S_t = S_{t-1} + \beta_t(v_t - S_{t-1}k_t)k_t^T$$

имеет сразу три interpretations:

1. correction текущей retrieval error;
2. erase old association + write new association;
3. один gradient descent step по local quadratic retrieval loss.

Именно этот механизм дальше будет комбинироваться с global forgetting.

6. Gated DeltaNet: две разные ручки управления памятью

Зачем эта тема здесь

Delta rule умеет адресно исправлять association вдоль текущего key.

Но представим другой случай.

Sequence долго обсуждала одну тему, после чего начинается совершенно новый document или новый semantic segment. Возможно, полезно быстро ослабить большой объём накопленного state, даже если текущий key не совпадает со всеми старыми key directions.

Delta rule не делает такой global reset автоматически.

Decay умеет его делать, но decay не умеет аккуратно заменить одну association, сохранив остальные.

Отсюда естественная идея: использовать оба механизма одновременно.

Именно это является основной мотивацией Gated DeltaNet: gating отвечает за rapid memory erasure, delta rule - за targeted updates. Так эту комплементарность формулируют и авторы работы.

Интуиция

У нас появляются две независимые decisions.

Первая:

сколько старого state вообще сохранить сейчас?

Для неё нужен α_t .

Вторая:

насколько сильно нужно скорректировать read по текущему direction k_t ?

Для неё нужен β_t .

Это действительно разные вопросы.

Если объединить их в одну ручку, model придётся одним scalar одновременно принимать решение о global memory lifetime и о локальной correction strength.

Никакой причины ожидать, что эти decisions всегда совпадают, нет.

Понятия, которые вводятся

Gated delta rule - delta update после data-dependent scaling старого state.

α_t - global retention/forgetting gate.

β_t - targeted write strength.

Global erasure - ослабление state независимо от конкретного key direction.

Selective update - correction association, адресованная через current key.

Формальное изложение

Проверка orientation и shapes

В orientation курса $S_t \in \mathbb{R}^{d_v \times d_k}$ gated delta rule имеет вид

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Эта запись согласована по размерностям.

Проверим буквально:

$$I - \beta_t k_t k_t^\top \in \mathbb{R}^{d_k \times d_k}.$$

Следовательно,

$$S_{t-1} (I - \beta_t k_t k_t^\top) \in \mathbb{R}^{d_v \times d_k}.$$

И $v_t k_t^\top \in \mathbb{R}^{d_v \times d_k}$.

Поэтому formula корректна даже при $d_k \neq d_v$.

Как получить formula из двух операций

Сначала применим global decay: $\tilde{S}_{t-1} = \alpha_t S_{t-1}$.

Теперь читаем old association уже из decayed memory: $\tilde{v}_t = \tilde{S}_{t-1} k_t$.

Применяем обычный delta update к $\tilde{S}_{t-1}$:

$$S_t = \tilde{S}_{t-1} + \beta_t (v_t - \tilde{v}_t) k_t^\top.$$

Подставляем $\tilde{S}_{t-1} = \alpha_t S_{t-1}$:

$$S_t = \alpha_t S_{t-1} + \beta_t (v_t - \alpha_t S_{t-1} k_t) k_t^\top.$$

Раскрываем скобку:

$$S_t = \alpha_t S_{t-1} + \beta_t v_t k_t^\top - \alpha_t \beta_t S_{t-1} k_t k_t^\top.$$

Выносим α_t :

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Это gated delta rule.

Что делает α_t

Если $\beta_t = 0$, то $S_t = \alpha_t S_{t-1}$.

Текущего write нет, остаётся только decay.

Если α_t близко к единице, старый state в целом сохраняется.

Если α_t близко к нулю, большая часть предыдущего state быстро стирается.

Это global control.

Что делает β_t

Если $\alpha_t = 1$, получаем обычный delta rule:

$$S_t = S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

При unit key read по k_t после update равен

$$S_t k_t = (1 - \beta_t) S_{t-1} k_t + \beta_t v_t.$$

Поэтому β_t управляет локальной replacement strength.

Exact overwrite сохраняется

Возьмём unit key: $\|k_t\|_2 = 1$.

Для Gated DeltaNet:

$$S_t k_t = \alpha_t (1 - \beta_t) S_{t-1} k_t + \beta_t v_t.$$

Если $\beta_t = 1$, первый term исчезает: $S_t k_t = v_t$.

То есть global decay не ломает exact correction по направлению текущего key.

Разложение на current key direction и всё остальное

Это самый наглядный способ увидеть разницу двух gates.

Пусть $\|k_t\|_2 = 1$.

Любой старый state можно разложить как $S_{t-1} = ak_t^\top + S_\perp$, где $a = S_{t-1}k_t$ и $S_\perp k_t = 0$.

Применяем delta transition: $S_{t-1}(I - \beta_t k_t k_t^\top)$.

Так как $k_t^\top k_t k_t^\top = k_t^\top$ и $S_\perp k_t k_t^\top = 0$, получаем

$$S_{t-1}(I - \beta_t k_t k_t^\top) = (1 - \beta_t)ak_t^\top + S_\perp.$$

Теперь добавляем global gate и write:

$$S_t = \alpha_t S_\perp + \alpha_t(1 - \beta_t)ak_t^\top + \beta_t v_t k_t^\top.$$

Собираем current key direction:

$$S_t = \alpha_t S_\perp + (\alpha_t(1 - \beta_t)a + \beta_t v_t)k_t^\top.$$

Теперь роли видны буквально.

α_t действует и на $S_\perp$, то есть на memory outside current key direction.

β_t определяет replacement внутри current key direction.

Поэтому эти параметры нельзя считать двумя названиями одного и того же gate.

Пример

Пусть $d_k = 2$, $d_v = 1$.

Current key:

$$k = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Старый state:

$$S_{t-1} = \begin{bmatrix} 10 & 5 \end{bmatrix}.$$

При $d_v = 1$ состояние - строка 1×2 . Первая component соответствует current key direction.

Вторая component ортогональна current key и может представлять другую старую информацию.

Хотим записать новый value $v = 2$.

Пусть $\alpha = 0.1$, $\beta = 1$.

Delta transition при $\beta = 1$:

$$I - kk^T = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

Удаляем current key component старого state:

$$S_{t-1}(I - kk^T) = [0 \quad 5].$$

Применяем global decay:

$$0.1 [0 \quad 5] = [0 \quad 0.5].$$

Пишем новый value:

$$vk^T = 2 \cdot [1 \quad 0] = [2 \quad 0].$$

Получаем

$$S_t = [2 \quad 0.5].$$

Что произошло:

- association по текущему key стала ровно **2**;
- unrelated old component уменьшилась с **5** до **0.5**.

Теперь уберём global gate, то есть положим $\alpha = 1$:

$$S_t = [2 \quad 5].$$

Current association исправлена, но старая orthogonal memory вообще не ослабла.

Теперь рассмотрим только scalar decay с обычным additive write: $S_t = \alpha S_{t-1} + vk^T$.

При $\alpha = 0.1$:

$$S_t = [3 \quad 0.5].$$

По current key читается **3**, а нужен **2**.

Чтобы сильнее удалить old value **10**, придётся ещё сильнее уменьшить α , но тогда одновременно исчезнет unrelated component **5**.

Именно этот конфликт и объясняет две ручки.

Где здесь легко ошибиться

«Можно оставить только α_t »

Можно построить gated additive model $S_t = \alpha_t S_{t-1} + v_t k_t^\top$.

Но scalar α_t не умеет удалить одну association, сохранив остальные.

Чтобы убрать старый content вдоль k_t , приходится ослаблять весь state.

«Можно оставить только β_t »

Это обычный DeltaNet при $\alpha_t = 1$.

Он умеет исправлять direction текущего key.

Но component state, которая почти не пересекается с будущими keys, может жить очень долго. У model нет отдельной ручки быстрого global forgetting.

«Если $\beta_t = 0$, Gated DeltaNet всё равно что-то запишет»

Нет.

При $\beta_t = 0$ остаётся только $S_t = \alpha_t S_{t-1}$.

«Если убрать learned β_t , ничего не потеряем»

Если фиксировать $\beta_t = 1$ при unit key, read по current key получает maximal exact correction, но неортогональные directions могут измениться.

Model теряет возможность:

- почти ничего не менять;
- делать partial correction;
- задавать input-dependent write strength.

« α_t и β_t оба просто learning rates»

Нет.

β_t действительно естественно интерпретируется как step size delta correction.

α_t масштабирует pre-existing state как global retention factor.

«Gated DeltaNet - это Mamba-2 плюс одна маленькая формула, значит architectures эквивалентны»

Нет.

Авторы сами мотивируют Gated DeltaNet комбинацией gating и delta rule, но full block включает projections, normalization, short convolution, output gating и другие choices. В published ICLR 2025 version q

и k L2-normalized, а hardware-efficient chunkwise algorithm является отдельной частью architecture.

Сходство state recurrences не является equality full networks.

Что теперь понятно

Delta rule и decay контролируют разные axes memory management.

Можно кратко запомнить так:

α_t : сколько прошлого **state** оставить вообще,

β_t : насколько сильно исправить **association** по **current key**.

Gated DeltaNet совмещает global forgetting с targeted correction.

Именно поэтому две ручки здесь содержательно оправданы, а не являются redundant parameterization.

7. Карта семейств: похожая нотация не означает одну architecture

Зачем эта тема здесь

Теперь у нас накопилось достаточно простых state equations, чтобы многие architectures начать выглядеть подозрительно похожими.

Почти везде можно увидеть что-то вроде

$$\text{new state} = \text{old state} \times \text{transition} + \text{write}.$$

Очень легко сделать следующий шаг:

значит RetNet, Mamba, RWKV и DeltaNet - это одно и то же.

Именно его делать нельзя.

Точная формулировка такая: доказаны equivalences конкретных operators или computational forms, а не всех полных architectures.

Нужна карта, на которой видно и сходство, и границы утверждения.

Интуиция

Сравнивать architectures полезно по слоям.

Первый уровень - **что является long-range operator**.

Это может быть:

- finite associative matrix state;
- SSM recurrence;
- long convolution;
- attention.

Второй - **как state или history управляется**.

Например:

- additive accumulation;
- scalar decay;
- data-dependent decay;
- delta correction;
- vector-valued gates.

Третий - **как operator вычисляется на hardware**.

Например:

- literal pairwise parallel form;
- recurrence;
- scan;
- chunkwise algorithm;
- FFT convolution.

Четвёртый - **full architecture вокруг operator**:

- projections;
- short convolutions;
- normalization;
- channel mixing;
- MLP;
- positional mechanism;
- residual structure;
- number и arrangement layers.

Совпадение на одном уровне не означает совпадения на остальных.

Понятия, которые вводятся

Selective SSM - state-space recurrence с input-dependent control.

State space duality, SSD - formal framework Mamba-2, связывающий определённый класс structured SSM и attention-like structured matrix transformations.

Generalized delta rule - семейство delta-like updates с более выразительными gates или erase/write directions.

Long convolution - long-range mixing через convolution kernel.

Implicit filter - compact parameterization convolution filter.

Формальное изложение

RetNet, 2023

В нашей упрощённой notation RetNet anchor выглядит как $S_t = \gamma S_{t-1} + v_t k_t^\top$.

Главная причина держать RetNet в этом занятии - retention operator с explicit parallel, recurrent и chunkwise recurrent representations. Это именно пример **разных algorithms одного operator**.

Из этого не следует equivalence RetNet с Mamba, RWKV или DeltaNet.

Mamba, 2023

Mamba строится вокруг selective SSM. Основная идея - сделать SSM parameters input-dependent, чтобы model могла content-dependently propagate или forget information.

Это содержательно ближе к data-dependent memory control, чем к fixed decay.

Но scalar recurrence $S_t = \gamma_t S_{t-1} + v_t k_t^\top$ не является определением Mamba.

Подробно selective SSM будет разбираться на занятии 3.

Mamba-2 / SSD, 2024

Работа Dao и Gu вводит Structured State Space Duality и через неё Mamba-2. Авторы показывают formal connections между SSM и attention variants через structured semiseparable matrices.

Это сильнее, чем просто архитектурная аналогия.

Но и здесь нужно точно назвать объект утверждения: речь о mathematical classes/operators и algorithms, а не о том, что произвольный Transformer block и полный Mamba-2 block являются одной и той же neural network.

DeltaNet, 2024

Современный DeltaNet anchor курса - работа про parallelization over sequence length:

$$S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Delta-like fast-weight memory была предложена раньше, в работе 2021 года. Работа 2024 года важна тем, что делает этот operator практически обучаемым через hardware-efficient chunkwise algorithm с WY representation.

Gated DeltaNet, 2024/2025

ArXiv version появилась в декабре 2024 года, published version - ICLR 2025.

В нашей orientation core recurrence:

$$S_t = \alpha_t S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Здесь совмещены scalar data-dependent decay и delta correction.

RWKV, historical 2023 anchor

Работа *RWKV: Reinventing RNNs for the Transformer Era* появилась в 2023 году и представляет ранний важный anchor recurrent/linear-attention line.

Но дальше фраза «RWKV делает X» без version number слишком неоднозначна.

RWKV family заметно менялась.

Поэтому после этого занятия корректнее говорить RWKV-6, RWKV-7 и так далее, если обсуждается конкретный update.

RWKV-7, 2025

RWKV-7 "Goose" уже прямо описывает свой mechanism как generalized delta rule с vector-valued gating и in-context learning rates. Paper опубликован на COLM 2025.

Это хороший пример эволюции идеи:

scalar delta control $\rightarrow$ more expressive vector-valued state control.

Но generalized delta rule RWKV-7 не тождественен scalar Gated DeltaNet.

Hyena, 2023

Hyena находится на другой ветке.

Её long-range operator построен из implicitly parametrized long convolutions и data-controlled gating.

Поэтому Hyena не нужно насильно записывать в таблицу как «ещё один delta rule».

Это long-convolution architecture.

Карта

Семейство	Anchor	Что является главным long-range mechanism	Что важно для этого занятия
Hyena	2023	implicit long convolution + data-controlled gating	Отдельная convolutional ветка, не delta memory.
RWKV paper	2023	recurrent/linear-attention family	Исторический anchor; дальше обязательно указывать version.
RetNet	2023	decayed retention	Чистый пример parallel/recurrent/chunkwise forms.
Mamba	2023	selective SSM	Data-dependent state-space dynamics; подробно занятие 3.
Mamba-2 / SSD	2024	structured SSM + SSD	Formal operator-level bridge между SSM и attention-like structured matrices.
DeltaNet	2024	delta-rule associative state	Targeted correction; hardware-efficient chunkwise parallelization.
Gated DeltaNet	arXiv 2024, ICLR 2025	global gate + delta rule	α_t и β_t управляют разными memory operations.
RWKV-7	2025	generalized delta rule	Vector-valued gating и более выразительная state evolution.

Где именно доказана эквивалентность, а где нет

Удобно снова разделить три уровня.

1. Равенство operators

Например, для fixed decay $Y = ((QK^\top) \odot R^{(\gamma)})V$ и

$$S_t = \gamma S_{t-1} + v_t k_t^\top, \quad y_t = S_t q_t$$

дают одинаковую функцию от Q, K, V .

Это точное algebraic statement.

2. Разные algorithms одного operator

Parallel, recurrent и chunkwise RetNet-style retention - именно такой случай.

DeltaNet тоже имеет exact recurrent и chunkwise algorithms, хотя derivation chunkwise form сложнее.

3. Сходство architecture principles

«В обеих моделях есть data-dependent forgetting» - полезное сравнение.

Но из него не следует equality operators.

Тем более из него не следует equality full architectures.

Mamba, RetNet, Gated DeltaNet и RWKV-7 используют разные state parameterizations, transitions, projections, normalization, local mixing и block structures.

SSD - более сильное утверждение, но всё равно не equality full architectures

SSD действительно устанавливает formal mathematical bridge между классами SSM и attention-like transformations. Это не просто метафора.

Но область theorem/framework должна сохраняться в формулировке.

Фраза

«Mamba и Transformer - одна architecture»

из SSD не следует.

Пример: один semantic event, разные mechanisms

Представим, что model встречает явную границу documents.

Decay-based mechanism может сделать gate маленьким и быстро ослабить прошлое: $\gamma_t \approx 0$.

Gated DeltaNet может сделать $\alpha_t \approx 0$ и одновременно записать новую association через β_t .

DeltaNet без global gate может корректировать associations, keys которых встречаются снова, но у него нет отдельного scalar operation «ослабить всё».

Selective SSM может реализовывать selective propagation/forgetting через свои input-dependent state-space parameters.

Hyena обрабатывает long-range dependency через convolutional operator и gates, поэтому описывать тот же event нужно уже в другой mathematical language.

Все architectures могут в итоге научиться похожему поведению на task level.

Из этого не следует, что внутренние operators одинаковы.

Hyena и autoregressive decode

Здесь есть отдельная ловушка.

Long convolution длины L формально выглядит как

$$y_t = \sum_{\tau=0}^{L-1} h_{\tau} x_{t-\tau}.$$

Если буквально пересчитывать эту сумму на каждом новом token, потребуется:

- хранить history;
- выполнять работу, растущую с effective filter length.

Но training long convolution можно эффективно вычислять другими algorithms, например convolution/FFT-style methods.

Теперь главный момент: **training representation long convolution ничего автоматически не говорит о стоимости token-by-token decode.**

Если конкретный filter допускает compact recurrent или state-space realization, можно поддерживать finite state и обновлять его на каждом token.

Если такого compact realization нет, implicit parameterization coefficients сама по себе ещё не гарантирует дешёвый exact recurrent decode.

Поэтому неверны обе слишком сильные формулировки:

«Hyena - long convolution, значит дешёвый autoregressive decode невозможен».

и

«Hyena filter параметризован компактно, значит decode автоматически constant-state».

Нужно смотреть на parameterization filter и наличие соответствующего compact recurrent representation.

Исходная Hyena работа определяет architecture через implicitly parametrized long convolutions и data-controlled gating. Из самого этого определения asymptotic decode cost без дополнительного анализа не следует.

Где здесь легко ошибиться

«Data-dependent γ - это Mamba»

Нет. Это общий design principle selective forgetting.

«Все linear recurrent architectures эквивалентны»

Нет. Даже state dimensions, transition class и write rule могут различаться.

«SSD доказал, что все SSM и attention одинаковы»

Нет. SSD устанавливает formal duality для определённого structured class.

«RWKV делает delta rule»

Нужно назвать version.

Для RWKV-7 такое утверждение осмысленно и прямо поддерживается paper. Для исторической RWKV 2023 это уже другая формулировка.

«Hyena - ещё один finite-state delta model»

Нет. Основной Hyena operator - gated long convolution.

«Fixed-size state решил проблему long context»

Лучше заменить на:

fixed-size recurrent state устраняет рост persistent state с prefix для данного operator, но переносит проблему в capacity, interference, memory control и качество long-range retrieval.

Что теперь понятно

Современные efficient sequence architectures действительно удобно приводить к близкому языку:

state, transition, write, read, gate.

Этот язык полезен для сравнения.

Но он является common notation, а не theorem об equality architectures.

Главная схема занятия

Начинаем с additive memory: $S_t = S_{t-1} + v_t k_t^\top$.

При чтении старого state current key видит все прошлые associations:

$$S_{t-1} k_t = \sum_{i=1}^{t-1} (k_t^\top k_i) v_i.$$

Нужный value смешивается с interference:

$$\hat{v} = v_j + \sum_{i \neq j} (k_j^\top k_i) v_i$$

для unit matched key.

State содержит фиксированные $d_k d_v$ scalars, а число write attempts растёт вместе с T .

Первое исправление - decay: $S_t = \gamma S_{t-1} + v_t k_t^\top$.

Unroll:

$$S_t = \sum_{i=1}^t \gamma^{t-i} v_i k_i^\top.$$

Data-dependent version: $S_t = \gamma_t S_{t-1} + v_t k_t^\top$,

$$S_t = \sum_{i=1}^t \left(\prod_{j=i+1}^t \gamma_j \right) v_i k_i^\top.$$

Decay управляет lifetime, но не умеет targeted replacement.

Delta rule сначала читает old value: $\hat{v}_t = S_{t-1}k_t$.

Затем пишет error: $S_t = S_{t-1} + \beta_t(v_t - \hat{v}_t)k_t^\top$.

Эквивалентная matrix form:

$$S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Эта формула является одним gradient descent step по

$$\ell_t(S) = \frac{1}{2} \|S k_t - v_t\|_2^2.$$

При unit key: $S_t k_t = (1 - \beta_t)\hat{v}_t + \beta_t v_t$.

При $\beta_t = 1$: $S_t k_t = v_t$.

Gated DeltaNet добавляет global retention:

$$S_t = \alpha_t S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Здесь:

α_t управляет **global forgetting**,

β_t управляет **targeted correction**.

Для fixed decay literal parallel form остаётся простой weighted causal matrix.

Для delta transition становится matrix-valued: $A_t = I - \beta_t k_t k_t^\top$.

Past contribution проходит через ordered products $A_{i+1}A_{i+2} \cdots A_t$, стоящие справа от записи.

Именно здесь efficient sequence parallelization перестаёт следовать из простой перестановки matrix products и требует отдельного algorithm.

Вопросы для самопроверки

1. Откуда возникает interference в additive state?

Ответ.

До текущего write

$$S_{t-1} = \sum_{i=1}^{t-1} v_i k_i^\top.$$

Read по key k_t :

$$S_{t-1} k_t = \sum_{i=1}^{t-1} (k_t^\top k_i) v_i.$$

Поэтому любой прошлый key с ненулевым $k_t^\top k_i$ добавляет свой value в output.

2. Почему почти ортогональные keys не гарантируют отсутствие interference?

Ответ.

Каждый отдельный dot product может быть мал, но число terms растёт.

В упрощённой random model squared noise scale растёт примерно как m/d_k , поэтому RMS interference - как $\sqrt{m/d_k}$.

3. Как сформулировать exact capacity limitation через linear algebra?

Ответ.

Для m точных associations нужно решить $K_m S^\top = V_m$.

При $m > d_k$ rank K_m не превосходит d_k , поэтому arbitrary V_m вообще говоря не лежит в image этого linear map.

Следовательно, arbitrary exact bindings невозможно наращивать без ограничений.

4. Почему нельзя сказать «capacity ровно d_k tokens»?

Ответ.

Потому что речь идёт о capacity для arbitrary independent exact key-value constraints.

Real sequence information может быть structured, compressed и распределена между несколькими heads/layers.

5. Разверните fixed-decay recurrence.

Ответ.

Из $S_t = \gamma S_{t-1} + v_t k_t^\top$ при $S_0 = 0$ получаем

$$S_t = \sum_{i=1}^t \gamma^{t-i} v_i k_i^\top.$$

6. Как выглядит data-dependent decay после unroll?

Ответ.

$$S_t = \sum_{i=1}^t \left(\prod_{j=i+1}^t \gamma_j \right) v_i k_i^\top.$$

Каждая старая запись проходит через все последующие retention gates.

7. Почему data-dependent γ_t ещё не делает operator Mamba?

Ответ.

Mamba - конкретный selective SSM с input-dependent SSM dynamics и определённой block/algorithmic structure.

Scalar input-dependent decay является только одним похожим design principle.

8. Как выглядит simplified RetNet-like parallel form?

Ответ.

Определяем

$$R_{ti}^{(\gamma)} = \mathbf{1}[i \leq t] \gamma^{t-i}.$$

Тогда $Y = ((QK^\top) \odot R^{(\gamma)})V$.

9. Что произойдёт с RetNet-like formulas при $\gamma = 1$?

Ответ.

Decayed mask превращается в ordinary inclusive causal mask: $R^{(1)} = M$.

Recurrent update превращается в additive state.

Chunkwise formula возвращается к формулам занятия 1.

10. Как изменится local objective, если текущему примеру дан weight $w_t > 0$?

Ответ.

Возьмём weighted local retrieval loss:

$$\ell_t^{(w)}(S) = \frac{1}{2} w_t \|S k_t - v_t\|_2^2.$$

Нужно определить, как weight изменяет gradient и effective correction strength.

11. Чему равен gradient weighted loss по S ?

Ответ. $\nabla_S \ell_t^{(w)} = w_t (S k_t - v_t) k_t^\top$.

Weight w_t масштабирует gradient, не меняя его shape $d_v \times d_k$.

12. Как weighted loss меняет delta update и условие exact correction для unit key?

Ответ. Один gradient step даёт

$$S_t = S_{t-1} + \beta_t w_t (v_t - S_{t-1} k_t) k_t^\top.$$

Effective correction strength равна $\beta_t w_t$. Для unit key exact correction по current key получается при $\beta_t w_t = 1$; другие неортогональные directions при этом могут измениться.

13. Почему при unit key и $\beta_t = 1$ происходит exact correction по направлению текущего key?

Ответ.

Для unit key $S_t k_t = (1 - \beta_t) \hat{v}_t + \beta_t v_t$.

При $\beta_t = 1$: $S_t k_t = v_t$.

14. Что меняется, если key не нормирован?

Ответ.

Correction coefficient становится $\beta_t \|k_t\|_2^2$.

Поэтому $\beta_t = 1$ уже не означает exact one-step overwrite.

15. Почему delta rule не является normalized linear attention?

Ответ.

Normalized linear attention делит read на scalar normalizer:

$$y_t = \frac{S_t q_t}{q_t^\top z_t + \epsilon}.$$

Delta rule изменяет сам S_t через prediction error: $v_t - S_{t-1} k_t$.

Это разные operators и разные memory dynamics.

16. Когда delta update не меняет read другого key u ?

Ответ.

Из

$$S_t u = S_{t-1} u + \beta_t (v_t - \hat{v}_t) (k_t^\top u)$$

видно, что read не меняется при $u^\top k_t = 0$.

17. Корректна ли formula Gated DeltaNet в orientation этого курса?

Ответ.

Да:

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Матрица в parentheses имеет shape $d_k \times d_k$, поэтому первый term и outer-product write имеют shape $d_v \times d_k$.

Равенство $d_k = d_v$ не требуется.

18. В чём разница между α_t и β_t ?

Ответ.

α_t масштабирует old state глобально.

β_t управляет correction association вдоль current key.

Они действуют на разные части state dynamics.

19. Что получится при $\alpha_t = 1$?

Ответ.

Обычный delta rule:

$$S_t = S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

20. Что получится при $\beta_t = 0$?

Ответ.

Только decay: $S_t = \alpha_t S_{t-1}$.

На этом шаге новая association не записывается.

21. Почему одной α_t недостаточно для targeted correction?

Ответ.

Scalar α_t ослабляет весь state.

Чтобы удалить старую association по одному key, приходится одновременно портить unrelated memory.

22. Почему одной β_t недостаточно для global forgetting?

Ответ.

Delta correction действует через current k_t .

State components, почти ортогональные всем будущим keys, могут сохраняться. Отдельного global reset нет.

23. Есть ли exact parallel/recurrent/chunkwise forms у decay operator?

Ответ.

Да.

Для fixed scalar decay parallel form использует mask γ^{t-i} , recurrent form умножает state на γ , chunkwise form объединяет cross-chunk recurrence и intra-chunk decayed pairwise computation.

24. Есть ли такие формы у delta operator?

Ответ.

Recurrent и exact chunkwise forms есть.

Formal pairwise representation тоже можно записать.

Но transition matrices $A_t = I - \beta_t k_t k_t^\top$ зависят от sequence и входят в ordered matrix products. Поэтому дешёвая fully parallel computation уже не получается простой перестановкой скобок.

25. Что именно решает fixed-size state в long-context problem?

Ответ.

Он может убрать линейный рост persistent state с prefix при recurrent decode конкретного operator.

Он не гарантирует:

- lossless memory;
- отсутствие interference;
- хороший long-range retrieval;
- constant workspace training;
- hardware-efficient sequence parallelism.

26. Почему RetNet, Mamba и RWKV нельзя назвать одной architecture?

Ответ.

Похожий общий шаблон recurrence не фиксирует transition parameterization, write/read rules, normalization, local mixing, projections, gates и block structure.

Нужно отдельно доказывать operator equality для конкретных formulas.

27. Что сильнее: similarity principle или SSD result?

Ответ.

Фраза «обе модели используют state decay» - architectural analogy.

SSD - formal mathematical framework, связывающий определённые structured SSM и attention-like operators.

Но даже SSD не означает identity произвольных full architectures.

28. Почему теперь нельзя просто говорить «RWKV»?

Ответ.

RWKV family существенно менялась.

В частности, RWKV-7 использует generalized delta rule с vector-valued gating и in-context learning rates. Поэтому statement о current mechanism должен сопровождаться version.

29. Почему Нуена не позволяет сделать вывод о decode cost только из слов «long convolution»?

Ответ.

Literal long convolution может требовать history-dependent work на каждом token.

No filter иногда может допускать compact recurrent/state-space realization.

С другой стороны, compact implicit parameterization coefficients ещё не гарантирует такой realization.

Decode cost нужно выводить из конкретного operator representation.

Что читать дальше

Imanol Schlag, Kazuki Irie, Jürgen Schmidhuber. *Linear Transformers Are Secretly Fast Weight Programmers*. ICML 2021. Читать ради conceptual bridge между linear attention, finite fast-weight memory, capacity problem и delta-like programming instruction. Это источник, где особенно хорошо видно, почему additive outer products плохо работают при повторном использовании memory.

Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, Yoon Kim. *Parallelizing Linear Transformers with the Delta Rule over Sequence Length*. NeurIPS 2024. Читать после понимания самой delta formula. Основная причина - увидеть, почему recurrent equation недостаточно: нужно отдельно решить проблему hardware-efficient training, и именно здесь появляются generalized Householder transformations, WY representation и chunkwise algorithm.

Songlin Yang, Jan Kautz, Ali Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. arXiv 2024, ICLR 2025. Читать ради разделения ролей gating и delta update. Центральный тезис статьи совпадает с главным выводом раздела 5: gating даёт rapid memory erasure, delta rule - targeted modification.

Yutao Sun et al. *Retentive Network: A Successor to Transformer for Large Language Models*. 2023.

Читать прежде всего sections про parallel, recurrent и chunkwise recurrent retention. Полную architecture для этого занятия восстанавливать не требуется.

Albert Gu, Tri Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2023. Это подготовка к занятию 3. Читать ради вопроса: что именно означает selectivity в SSM и почему input-dependent forgetting - более широкая идея, чем scalar γ_t .

Tri Dao, Albert Gu. *Transformers are SSMS: Generalized Models and Efficient Algorithms Through Structured State Space Duality*. 2024. Основное чтение к следующему занятию. Нужна не только Mamba-2 architecture, но и SSD как аккуратный пример того, как доказывать связь classes of operators, не превращая её в утверждение «все architectures одинаковы».

Bo Peng et al. *RWKV-7 "Goose" with Expressive Dynamic State Evolution*. COLM 2025.

Дополнительное чтение после delta rule: посмотреть, как scalar delta mechanism развивается в generalized rule с vector-valued control.

Michael Poli et al. *Hyena Hierarchy: Towards Larger Convolutional Language Models*. ICML 2023.

Читать для сохранения второй ветки карты: efficient long-range operator необязательно обязан быть associative matrix state. Hyena строится через implicit long convolutions и data-controlled gating.

Что этот конспект добавляет к контракту нотации занятия 1

Все обозначения занятия 1 сохраняются буквально.

Не меняются:

$$q_t, k_t \in \mathbb{R}^{d_k}, \quad v_t \in \mathbb{R}^{d_v},$$

$$S_t \in \mathbb{R}^{d_v \times d_k},$$

$$y_t = S_t q_t,$$

а также [B, H, T, D], inclusive causality и отсутствие предположения $d_k = d_v$.

Добавляются следующие symbols.

Старое значение по current key

$$\hat{v}_t = S_{t-1} k_t \in \mathbb{R}^{d_v}.$$

Эквивалентная row notation: $\hat{v}_t^\top = k_t^\top S_{t-1}^\top$.

Это internal read **до** delta write.

Fixed retention factor

$$\gamma \in [0, 1].$$

Decay recurrence: $S_t = \gamma S_{t-1} + v_t k_t^\top$.

Data-dependent retention factor

$$\gamma_t \in [0, 1].$$

Update: $S_t = \gamma_t S_{t-1} + v_t k_t^\top$.

Retention mask

Для fixed decay:

$$R_{ti}^{(\gamma)} = \mathbf{1}[i \leq t] \gamma^{t-i}.$$

Delta write strength

$$\beta_t \in [0, 1]$$

в основной interpretation занятия.

Delta update: $S_t = S_{t-1} + \beta_t (v_t - \hat{v}_t) k_t^\top$.

Matrix form:

$$S_t = S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Delta transition

$$A_t = I - \beta_t k_t k_t^\top \in \mathbb{R}^{d_k \times d_k}.$$

Gated DeltaNet retention gate

$$\alpha_t \in [0, 1].$$

Gated delta update в orientation курса:

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Порядок операций для delta-like updates

На position t :

1. из S_{t-1} читается old association по k_t ;
2. вычисляется correction;
3. строится новый S_t ;
4. обычный output read вычисляется как $y_t = S_t q_t$.

То есть inclusive causal semantics занятия 1 сохраняется.

Переход к следующему вопросу курса

После двух занятий уже видно, что слово «линейный» скрывает несколько разных задач.

Additive linear attention дал compact recurrent state.

Decay добавил lifetime.

Delta rule добавил editing.

Gated delta rule разделил global forgetting и targeted correction.

Но возникает новый computational вопрос.

Для additive state transition тривиальна: $S_t = S_{t-1} + \text{write}$.

Для fixed decay transition остаётся scalar: $S_t = \gamma S_{t-1} + \text{write}$.

Для delta rule transition уже matrix-valued и input-dependent:

$$S_t = S_{t-1} A_t + \text{write}, \quad A_t = I - \beta_t k_t k_t^\top.$$

Чтобы эффективно обрабатывать целую sequence, нужно понимать structure products таких transitions и искать representations, которые одновременно дают:

- expressive state dynamics;
- cheap recurrent decode;

- достаточно parallel training.

Отсюда естественно перейти к state space models, selective dynamics и SSD.

То есть следующий вопрос курса уже не только

«как редактировать конечную память?»

а

«какие классы state transitions допускают одновременно выразительную dynamics и эффективные parallel algorithms?»